

Skill Session 17

Task 1

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

```
j_rakshitha@RAKSHITHA: ~  
j_rakshitha@RAKSHITHA:~$ nano session17_task1_jobs.c  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session17_task1_jobs.c -o session17_task1_jobs  
j_rakshitha@RAKSHITHA:~$ ./session17_task1_jobs  
Practical Skill Session 17 - Task 1  
=====  
Commands:  
  run <seconds> - start a background job  
  jobs          - list jobs  
  update        - update job states  
  cleanup       - remove completed jobs  
  exit          - quit  
  
job-shell> run 10  
Started job [1] with PID 745  
  
job-shell> jobs  
  
Active Jobs  
=====
```

[1]	PID=745	RUNNING	run 10
-----	---------	---------	--------

```
=====
```

job-shell> run 5
Started job [2] with PID 748

job-shell> jobs

Active Jobs

```
=====
```

[1]	PID=745	COMPLETED	run 10
[2]	PID=748	RUNNING	run 5

```
=====
```

job-shell> update
Job states updated.

job-shell> jobs

Active Jobs

```
=====
```

[1]	PID=745	COMPLETED	run 10
[2]	PID=748	COMPLETED	run 5

```
=====
```

job-shell> cleanup
Completed jobs removed.

job-shell> jobs
No jobs.

job-shell> exit
j_rakshitha@RAKSHITHA:~\$ |

j_rakshitha@RAKSHITHA: ~



GNU nano 8.7.1 session17_task1_jobs.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef enum
{
    RUNNING,
    COMPLETED
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strncpy(jobs[job_count].command,
            command,
            sizeof(jobs[job_count].command) - 1);

    jobs[job_count].command[
        sizeof(jobs[job_count].command) - 1
    ] = '\0';

    jobs[job_count].state = RUNNING;

    job_count++;
}

void update_jobs(void)
{

```

```
for (int i = 0; i < job_count; i++)
{
    if (jobs[i].state == RUNNING)
    {
        pid_t result = waitpid(jobs[i].pid, NULL, WNOHANG);

        if (result == jobs[i].pid)
        {
            jobs[i].state = COMPLETED;
        }
    }
}

void list_jobs(void)
{
    update_jobs();

    if (job_count == 0)
    {
        printf("No jobs.\n");
        return;
    }

    printf("\nActive Jobs\n");
    printf("=====\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d %-10s %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].state == RUNNING ? "RUNNING" : "COMPLETED",
            jobs[i].command);
    }

    printf("=====\n");
}

void cleanup_jobs(void)
{
    int new_count = 0;

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == RUNNING)
        {
            jobs[new_count++] = jobs[i];
        }
    }
}
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session17_task1_jobs.c
    job_count = new_count;
}

int main(void)
{
    char input[100];

    printf("Practical Skill Session 17 - Task 1\n");
    printf("=====\n");

    printf("Commands:\n");
    printf("  run <seconds>  - start a background job\n");
    printf("  jobs           - list jobs\n");
    printf("  update         - update job states\n");
    printf("  cleanup        - remove completed jobs\n");
    printf("  exit          - quit\n");

    while (1)
    {
        printf("\njob-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strcmp(input, "jobs") == 0)
        {
            list_jobs();
        }
        else if (strcmp(input, "update") == 0)
        {
            update_jobs();
            printf("Job states updated.\n");
        }
        else if (strcmp(input, "cleanup") == 0)
        {
            cleanup_jobs();
            printf("Completed jobs removed.\n");
        }
        else if (strncmp(input, "run ", 4) == 0)
        {
            int seconds = atoi(input + 4);
```

```
j_rakshitha@RAKSHITHA: ~ × + v
GNU nano 8.7.1 session17_task1_jobs.c
    list_jobs();
}
else if (strcmp(input, "update") == 0)
{
    update_jobs();
    printf("Job states updated.\n");
}
else if (strcmp(input, "cleanup") == 0)
{
    cleanup_jobs();
    printf("Completed jobs removed.\n");
}
else if (strncmp(input, "run ", 4) == 0)
{
    int seconds = atoi(input + 4);

    if (seconds <= 0)
    {
        printf("Invalid duration.\n");
        continue;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        continue;
    }

    if (pid == 0)
    {
        sleep(seconds);
        exit(0);
    }

    add_job(pid, input);

    printf("Started job [%d] with PID %d\n",
           job_count,
           pid);
}
else
{
    printf("Unknown command.\n");
}
}

return 0;
}
```

Task 2

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
j_rakshitha@RAKSHITHA:~$ nano session17_task2_foreground.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session17_task2_foreground.c -o session17_task2_
foreground
j_rakshitha@RAKSHITHA:~$ ./session17_task2_foreground
Practical Skill Session 17 - Task 2
=====
Commands:
  run <seconds>  - start background job
  jobs           - list jobs
  fg <job_id>    - bring job to foreground
  exit          - quit

fg-shell> run 10
Started job [1] PID=917 PGID=917

fg-shell> Background job running for 10 seconds...
Background job finished.
jobs

Job List
=====
[1] PID=917 PGID=917 COMPLETED  duration=10 sec
=====

fg-shell> run 5
Started job [2] PID=947 PGID=947

fg-shell> Background job running for 5 seconds...
Background job finished.
fg 2
Job [2] is already completed.

fg-shell> run 15
Started job [3] PID=956 PGID=956

fg-shell> Background job running for 15 seconds...
fg 3

Bringing job [3] to foreground...
Transferring terminal control to PGID 956
Sending SIGCONT to job [3]
Waiting for foreground job to finish...
Background job finished.
Job [3] completed.
Restoring terminal control to shell...

fg-shell> exit
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session17_task2_foreground.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

#define MAX_JOBS 10

typedef enum
{
    RUNNING,
    COMPLETED
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    int duration;
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

pid_t shell_pgid;

void add_job(pid_t pid, pid_t pgid, int duration)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pgid;
    jobs[job_count].duration = duration;
    jobs[job_count].state = RUNNING;

    job_count++;
}

void update_jobs(void)
{
    for (int i = 0; i < job_count; i++)
    {

```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session17_task2_foreground.c *
    if (jobs[i].state == RUNNING)
    {
        pid_t result = waitpid(
            jobs[i].pid,
            NULL,
            WNOHANG
        );

        if (result == jobs[i].pid)
        {
            jobs[i].state = COMPLETED;
        }
    }
}

void list_jobs(void)
{
    update_jobs();

    if (job_count == 0)
    {
        printf("No jobs.\n");
        return;
    }

    printf("\nJob List\n");
    printf("=====\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d PGID=%d %-10s duration=%d sec\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].pgid,
            jobs[i].state == RUNNING
                ? "RUNNING"
                : "COMPLETED",
            jobs[i].duration);
    }

    printf("=====\n");
}

void foreground_job(int job_id)
{
    update_jobs();

    int index = -1;

    for (int i = 0; i < job_count; i++)
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo

j_rakshitha@RAKSHITHA: ~

GNU nano 8.7.1 session17_task2_foreground.c *

```
    if (jobs[i].job_id == job_id)
    {
        index = i;
        break;
    }
}

if (index == -1)
{
    printf("Job [%d] not found.\n", job_id);
    return;
}

if (jobs[index].state == COMPLETED)
{
    printf("Job [%d] is already completed.\n", job_id);
    return;
}

pid_t pgid = jobs[index].pgid;

printf("\nBringing job [%d] to foreground...\n", job_id);
printf("Transferring terminal control to PGID %d\n", pgid);

if (tcsetpgrp(STDIN_FILENO, pgid) == -1)
{
    perror("tcsetpgrp");
    return;
}

printf("Sending SIGCONT to job [%d]\n", job_id);

if (kill(-pgid, SIGCONT) == -1)
{
    perror("kill");
}

printf("Waiting for foreground job to finish...\n");

int status;

if (waitpid(
    jobs[index].pid,
    &status,
    0) == -1)
{
    perror("waitpid");
}

jobs[index].state = COMPLETED;
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session17_task2_foreground.c *

    printf("Job [%d] completed.\n", job_id);

    printf("Restoring terminal control to shell...\n");

    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
    {
        perror("tcsetpgrp");
    }
}

int main(void)
{
    char input[100];

    shell_pgid = getpid();

    /*
     * Put this shell into its own process group.
     */
    setpgid(0, shell_pgid);

    /*
     * Ignore terminal job-control signals in the shell.
     */
    signal(SIGTTOU, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTSTP, SIG_IGN);

    tcsetpgrp(STDIN_FILENO, shell_pgid);

    printf("Practical Skill Session 17 - Task 2\n");
    printf("=====\n");

    printf("Commands:\n");
    printf("  run <seconds>  - start background job\n");
    printf("  jobs           - list jobs\n");
    printf("  fg <job_id>    - bring job to foreground\n");
    printf("  exit           - quit\n");

    while (1)
    {
        printf("\nfg-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session17_task2_foreground.c *

    if (strcmp(input, "exit") == 0)
    {
        break;
    }

    if (strcmp(input, "jobs") == 0)
    {
        list_jobs();
    }
else if (strncmp(input, "run ", 4) == 0)
{
    int seconds = atoi(input + 4);

    if (seconds <= 0)
    {
        printf("Invalid duration.\n");
        continue;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        continue;
    }

    if (pid == 0)
    {
        /*
         * Child becomes a new process group leader.
         */
        setpgid(0, 0);

        printf("Background job running for %d seconds...\n",
               seconds);

        sleep(seconds);

        printf("Background job finished.\n");

        exit(0);
    }

    /*
     * Parent places child in its own process group.
     */
    setpgid(pid, pid);

    add_job(pid, pid, seconds);|

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```

        printf("Started job [%d] PID=%d PGID=%d\n",
               job_count,
               pid,
               pid);
    }
    else if (strncmp(input, "fg ", 3) == 0)
    {
        int job_id = atoi(input + 3);

        if (job_id <= 0)
        {
            printf("Invalid job ID.\n");
            continue;
        }

        foreground_job(job_id);
    }
    else
    {
        printf("Unknown command.\n");
    }
}

/*
 * Restore terminal control before exiting.
 */
tcsetpgrp(STDIN_FILENO, getpgrp());

return 0;
}

```

Skill Session 18

Task 1

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```
j_rakshitha@RAKSHITHA: ~  
j_rakshitha@RAKSHITHA:~$ nano session18_task1_signals.c  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session18_task1_signals.c -o session18_task1_signals  
j_rakshitha@RAKSHITHA:~$ ./session18_task1_signals  
Practical Skill Session 18 - Task 1  
=====
```

Commands:	
info	- show PID and process group
mask	- show signal mask
block	- block SIGUSR1
unblock	- unblock SIGUSR1
pending	- show pending signals
signal	- send SIGUSR1 to this process
exit	- quit

```
Press Ctrl+C to test terminal SIGINT.  
  
Process Information  
=====
```

Process PID :	1084
Process PGID:	1084
Parent PID :	326

```
=====
```

```
signal-shell> ^C  
Received SIGINT (Ctrl+C)  
SIGINT count: 1  
mask
```

```
Signal Mask Status  
=====
```

SIGINT :	NOT BLOCKED
SIGUSR1:	NOT BLOCKED
SIGTERM:	NOT BLOCKED

```
=====
```

```
signal-shell> block  
SIGUSR1 is now BLOCKED.
```

```
signal-shell> mask
```

```
Signal Mask Status  
=====
```

SIGINT :	NOT BLOCKED
SIGUSR1:	BLOCKED
SIGTERM:	NOT BLOCKED

```
=====
```

```
signal-shell> signal
Sending SIGUSR1 to process 1084...

signal-shell> signal
pendingSending SIGUSR1 to process 1084...

signal-shell> pending
Unknown command.

signal-shell> pending

Pending Signals
=====
SIGINT : NOT PENDING
SIGUSR1: PENDING
SIGTERM: NOT PENDING
=====

signal-shell> unblock

Received SIGUSR1
SIGUSR1 count: 1
SIGUSR1 is now UNBLOCKED.

signal-shell> mask

Signal Mask Status
=====
SIGINT : NOT BLOCKED
SIGUSR1: NOT BLOCKED
SIGTERM: NOT BLOCKED
=====

signal-shell> unblock
SIGUSR1 is now UNBLOCKED.

signal-shell> info

Process Information
=====
Process PID : 1084
Process PGID: 1084
Parent PID  : 326
=====
```

```
j_rakshitha@RAKSHITHA: ~  
GNU nano 8.7.1 session18_task1_signals.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <signal.h>  
#include <string.h>  
  
volatile sig_atomic_t sigint_count = 0;  
volatile sig_atomic_t sigusr1_count = 0;  
volatile sig_atomic_t sigterm_received = 0;  
  
void handle_sigint(int sig)  
{  
    (void)sig;  
    sigint_count++;  
  
    printf("\nReceived SIGINT (Ctrl+C)\n");  
    printf("SIGINT count: %d\n", sigint_count);  
    fflush(stdout);  
}  
  
void handle_sigusr1(int sig)  
{  
    (void)sig;  
    sigusr1_count++;  
  
    printf("\nReceived SIGUSR1\n");  
    printf("SIGUSR1 count: %d\n", sigusr1_count);  
    fflush(stdout);  
}  
  
void handle_sigterm(int sig)  
{  
    (void)sig;  
  
    sigterm_received = 1;  
  
    printf("\nReceived SIGTERM\n");  
    printf("Terminating safely...\n");  
    fflush(stdout);  
}  
  
void show_process_group(void)  
{  
    printf("\nProcess Information\n");  
    printf("=====\n");  
    printf("Process PID : %d\n", getpid());  
    printf("Process PGID: %d\n", getpgrp());  
    printf("Parent PID : %d\n", getppid());  
    printf("=====\n");  
}  
  
[ Read 258 lines ]  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session18_task1_signals.c

void show_signal_mask(void)
{
    sigset_t current_mask;

    if (sigprocmask(SIG_BLOCK, NULL, &current_mask) == -1)
    {
        perror("sigprocmask");
        return;
    }

    printf("\nSignal Mask Status\n");
    printf("=====\n");

    printf("SIGINT : %s\n",
        sigismember(&current_mask, SIGINT)
        ? "BLOCKED"
        : "NOT BLOCKED");

    printf("SIGUSR1: %s\n",
        sigismember(&current_mask, SIGUSR1)
        ? "BLOCKED"
        : "NOT BLOCKED");

    printf("SIGTERM: %s\n",
        sigismember(&current_mask, SIGTERM)
        ? "BLOCKED"
        : "NOT BLOCKED");

    printf("=====\n");
}

void block_sigusr1(void)
{
    sigset_t set;

    sigemptyset(&set);
    sigaddset(&set, SIGUSR1);

    if (sigprocmask(SIG_BLOCK, &set, NULL) == -1)
    {
        perror("sigprocmask");
        return;
    }

    printf("SIGUSR1 is now BLOCKED.\n");
}

void unblock_sigusr1(void)
{
    sigset_t set;

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

j_rakshitha@RAKSHITHA: ~



GNU nano 8.7.1

session18_task1_signals.c

```
sigemptyset(&set);
sigaddset(&set, SIGUSR1);

if (sigprocmask(SIG_UNBLOCK, &set, NULL) == -1)
{
    perror("sigprocmask");
    return;
}

printf("SIGUSR1 is now UNBLOCKED.\n");
}

void show_pending_signals(void)
{
    sigset_t pending;

    if (sigpending(&pending) == -1)
    {
        perror("sigpending");
        return;
    }

    printf("\nPending Signals\n");
    printf("=====\n");

    printf("SIGINT : %s\n",
           sigismember(&pending, SIGINT)
           ? "PENDING"
           : "NOT PENDING");

    printf("SIGUSR1: %s\n",
           sigismember(&pending, SIGUSR1)
           ? "PENDING"
           : "NOT PENDING");

    printf("SIGTERM: %s\n",
           sigismember(&pending, SIGTERM)
           ? "PENDING"
           : "NOT PENDING");

    printf("=====\n");
}

void send_sigusr1(void)
{
    printf("Sending SIGUSR1 to process %d...\n", getpid());

    if (kill(getpid(), SIGUSR1) == -1)
    {
        perror("kill");
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session18_task1_signals.c
}
}
void install_handlers(void)
{
    struct sigaction sa;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);

    sa.sa_handler = handle_sigint;
    sa.sa_flags = SA_RESTART;
    if (sigaction(SIGINT, &sa, NULL) == -1)
    {
        perror("sigaction SIGINT");
        exit(1);
    }

    sa.sa_handler = handle_sigusr1;
    sa.sa_flags = SA_RESTART;
    if (sigaction(SIGUSR1, &sa, NULL) == -1)
    {
        perror("sigaction SIGUSR1");
        exit(1);
    }

    sa.sa_handler = handle_sigterm;
    sa.sa_flags = SA_RESTART;
    if (sigaction(SIGTERM, &sa, NULL) == -1)
    {
        perror("sigaction SIGTERM");
        exit(1);
    }
}

int main(void)
{
    char input[100];

    install_handlers();

    printf("Practical Skill Session 18 - Task 1\n");
    printf("=====\n");

    printf("Commands:\n");
    printf("  info      - show PID and process group\n");
    printf("  mask      - show signal mask\n");
    printf("  block     - block SIGUSR1\n");
    printf("  unblock   - unblock SIGUSR1\n");
    printf("  pending   - show pending signals\n");
    printf("  signal    - send SIGUSR1 to this process\n");
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session18_task1_signals.c
printf("  exit      - quit\n");
printf("\nPress Ctrl+C to test terminal SIGINT.\n");

show_process_group();

while (!sigterm_received)
{
    printf("\nsignal-shell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "info") == 0)
    {
        show_process_group();
    }
    else if (strcmp(input, "mask") == 0)
    {
        show_signal_mask();
    }
    else if (strcmp(input, "block") == 0)
    {
        block_sigusr1();
    }
    else if (strcmp(input, "unblock") == 0)
    {
        unblock_sigusr1();
    }
    else if (strcmp(input, "pending") == 0)
    {
        show_pending_signals();
    }
    else if (strcmp(input, "signal") == 0)
    {
        send_sigusr1();
    }
    else if (strcmp(input, "exit") == 0)
    {
        break;
    }
    else if (strlen(input) > 0)
    {
        printf("Unknown command.\n");
    }
}

|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
    printf("\nProgram exiting.\n");
    return 0;
}
|
```

Task 2

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

```
j_rakshitha@RAKSHITHA: ~  
j_rakshitha@RAKSHITHA:~$ nano session18_task2_background.c  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session18_task2_background.c -o session18_task2_background  
j_rakshitha@RAKSHITHA:~$ ./session18_task2_background  
Practical Skill Session 18 - Task 2  
=====
```

Commands:

- run <seconds> - start a background job
- stop <job_id> - stop a running job
- bg <job_id> - resume a stopped job
- jobs - show job table
- exit - quit

```
job-shell> run 8  
Started job [1] PID=1275 PGID=1275  
  
job-shell> Job process 1275: working... 1/8  
Job process 1275: working... 2/8  
Job process 1275: working... 3/8  
Job process 1275: working... 4/8  
Job process 1275: working... 5/8  
Job process 1275: working... 6/8  
Job process 1275: working... 7/8  
Job process 1275: working... 8/8  
Job process 1275: finished.  
stop 1  
Job [1] is not running.  
  
job-shell> jobs  
  
Job Table  
=====
```

[1]	PID=1275	PGID=1275	STATE=COMPLETED
-----	----------	-----------	-----------------

```
=====
```

```
job-shell> run 20  
Started job [2] PID=1278 PGID=1278  
  
job-shell> Job process 1278: working... 1/20  
Job process 1278: working... 2/20  
Job process 1278: working... 3/20  
Job process 1278: working... 4/20  
stop 2  
Job [2] stopped using SIGSTOP.
```

```
j_rakshitha@RAKSHITHA: ~  
Job process 1278: working... 2/20  
Job process 1278: working... 3/20  
Job process 1278: working... 4/20  
stop 2  
Job [2] stopped using SIGSTOP.  
  
job-shell> jobs  
  
Job Table  
=====
```

[1]	PID=1275	PGID=1275	STATE=COMPLETED
[2]	PID=1278	PGID=1278	STATE=STOPPED

```
=====
```

```
job-shell> bg 2  
Sending SIGCONT to job [2]...  
Job [2] resumed successfully.  
  
job-shell> Job process 1278: working... 5/20  
Job process 1278: working... 6/20  
Job process 1278: working... 7/20  
Job process 1278: working... 8/20  
jobs  
  
Job Table  
=====
```

[1]	PID=1275	PGID=1275	STATE=COMPLETED
[2]	PID=1278	PGID=1278	STATE=RUNNING

```
=====
```

```
job-shell> Job process 1278: working... 9/20  
Job process 1278: working... 10/20  
Job process 1278: working... 11/20  
Job process 1278: working... 12/20  
Job process 1278: working... 13/20  
Job process 1278: working... 14/20  
Job process 1278: working... 15/20  
Job process 1278: working... 16/20  
Job process 1278: working... 17/20  
Job process 1278: working... 18/20  
Job process 1278: working... 19/20  
Job process 1278: working... 20/20  
Job process 1278: finished.  
jobs  
  
Job Table  
=====
```

[1]	PID=1275	PGID=1275	STATE=COMPLETED
[2]	PID=1278	PGID=1278	STATE=COMPLETED

```
=====
```

```
job-shell> exit  
Exiting.  
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session18_task2_background.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

typedef enum {
    RUNNING,
    STOPPED,
    COMPLETED
} JobState;

typedef struct {
    int id;
    pid_t pid;
    pid_t pgid;
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

const char *state_name(JobState state)
{
    switch (state) {
        case RUNNING:
            return "RUNNING";
        case STOPPED:
            return "STOPPED";
        case COMPLETED:
            return "COMPLETED";
        default:
            return "UNKNOWN";
    }
}

void update_jobs(void)
{
    int status;

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].state == COMPLETED)
            continue;

        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG | WUNTRACED);

        if (result == jobs[i].pid) {
            if (WIFSTOPPED(status)) {
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session18_task2_background.c
        jobs[i].state = STOPPED;
    } else if (WIFEXITED(status) || WIFSIGNALED(status)) {
        jobs[i].state = COMPLETED;
    }
}
}
}

void show_jobs(void)
{
    update_jobs();

    printf("\nJob Table\n");
    printf("=====\n");

    if (job_count == 0) {
        printf("No jobs.\n");
        printf("=====\n");
        return;
    }

    for (int i = 0; i < job_count; i++) {
        printf("[%d] PID=%d PGID=%d STATE=%s\n",
            jobs[i].id,
            jobs[i].pid,
            jobs[i].pgid,
            state_name(jobs[i].state));
    }

    printf("=====\n");
}

void run_job(int seconds)
{
    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        setpgid(0, 0);

        for (int i = 1; i <= seconds; i++) {
            printf("Job process %d: working... %d/%d\n",
                pid, i, seconds);
        }
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session18_task2_background.c
        getpid(), i, seconds);
        fflush(stdout);
        sleep(1);
    }

    printf("Job process %d: finished.\n", getpid());
    fflush(stdout);

    exit(0);
}

setpgid(pid, pid);

jobs[job_count].id = job_count + 1;
jobs[job_count].pid = pid;
jobs[job_count].pgid = pid;
jobs[job_count].state = RUNNING;

printf("Started job [%d] PID=%d PGID=%d\n",
        jobs[job_count].id,
        pid,
        pid);

    job_count++;
}

void stop_job(int id)
{
    update_jobs();

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].id == id) {
            if (jobs[i].state != RUNNING) {
                printf("Job [%d] is not running.\n", id);
                return;
            }

            if (kill(-jobs[i].pgid, SIGSTOP) == -1) {
                perror("SIGSTOP");
                return;
            }

            jobs[i].state = STOPPED;

            printf("Job [%d] stopped using SIGSTOP.\n", id);
            return;
        }
    }

    printf("Job [%d] not found.\n", id);
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session18_task2_background.c
void resume_job(int id)
{
    update_jobs();

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].id == id) {
            if (jobs[i].state != STOPPED) {
                printf("Job [%d] is not stopped.\n", id);
                return;
            }

            printf("Sending SIGCONT to job [%d]...\n", id);

            if (kill(-jobs[i].pgid, SIGCONT) == -1) {
                perror("SIGCONT");
                return;
            }

            jobs[i].state = RUNNING;

            printf("Job [%d] resumed successfully.\n", id);
            return;
        }
    }

    printf("Job [%d] not found.\n", id);
}

int main(void)
{
    char input[100];
    int seconds;
    int job_id;

    printf("Practical Skill Session 18 - Task 2\n");
    printf("=====\n");
    printf("Commands:\n");
    printf("  run <seconds>  - start a background job\n");
    printf("  stop <job_id>  - stop a running job\n");
    printf("  bg <job_id>    - resume a stopped job\n");
    printf("  jobs           - show job table\n");
    printf("  exit           - quit\n");

    while (1) {
        printf("\njob-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (sscanf(input, "run %d", &seconds) == 1) {
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
        if (seconds <= 0) {
            printf("Seconds must be greater than 0.\n");
        } else {
            run_job(seconds);
        }
    }
    else if (sscanf(input, "stop %d", &job_id) == 1) {
        stop_job(job_id);
    }
    else if (sscanf(input, "bg %d", &job_id) == 1) {
        resume_job(job_id);
    }
    else if (strcmp(input, "jobs\n") == 0) {
        show_jobs();
    }
    else if (strcmp(input, "exit\n") == 0) {
        break;
    }
    else {
        printf("Unknown command.\n");
    }
}

printf("\nExiting.\n");
return 0;
```

```
}
```

[Cancelled]